주차 진행 보고

LLM 기반 DAST 하네스

위험 기반 탐지 레벨 도입 · FR-04 통합 실행 · 속도 개선

미탐·오탐 해결 › FR-04 통합 › 속도 대응

2026-10 · 작성자: 박근령

이번 주 진행 요약

①

탐지 레벨 도입

미탐/오탐 문제를 '위험 기반 레벨'로 해결

②

FR-01~04 통합 실행

크롤링→선택→스캔→판단 1회 완주

③

속도 문제 식별·대응

스캔 시간 병목 확인 + 단축 적용

① 문제 — 미탐(놓침) vs 오탐

- **보안 진단의 핵심 2축: 일관성(재현성) · 보수적 탐지(놓치지 않기)**
- LLM 판단은 모델·호출마다 흔들리고(비결정성), 때로 치명적 취약점을 '안전'으로 통과(미탐)

지난주 발견 — `sort` 파라미터 미탐 사례

- 가벼운 모델(flash-lite): "sort는 표시용" 이라며 스캔 생략 — 신뢰도 0.95
- 실제로는 ORDER BY 절에 쓰여 SQL Injection에 취약 (고전적 벡터)
- **→ FR-02에서 skip하면 FR-03이 아예 안 봄 → 실제 취약점이 영구 미탐**
- 교훈: LLM의 높은 신뢰도 ≠ 정답. 미탐은 진단에서 가장 위험한 실패.

① 해결 — 위험 기반 탐지 레벨 도입

미탐↔오탐은 트레이드오프 → 운영자가 진단 시작 시 '레벨'을 선택해 조절

레벨	성격	동작	트레이드오프
Lv.3 보수적	미탐 최소	의심되면 전부 스캔 · 낮은 신뢰도도 확정	오탐·시간 ↑
Lv.2 균형	기본	위험 파라미터는 강제 포함	중간
Lv.1 정밀	오탐 최소	LLM 판단 존중 · 명확한 것만	미탐 ↑

산업 표준 개념: sqlmap --level/--risk, OWASP ZAP attack strength와 동일. 금융·공공은 레벨↑(보수적), 내부·관리자 페이지는 레벨↓(정밀)로 운영.

① 레벨 작동 방식 — 2중 구조

① 프롬프트 주입 (LLM 유도)

- 레벨별 지침을 LLM 프롬프트에 삽입
- Lv3: "의심되면 반드시 포함, 놓치지 마라"
- Lv1: "명확한 것만, 애매하면 생략"
- → LLM이 스스로 레벨에 맞춰 판단

② 규칙 가드레일 (코드 강제)

- 위험 파라미터(id·sort 등)는 강제 스캔
- 확정 문턱을 레벨별로 조절(FR-04)
- → LLM이 놓쳐도 구조적으로 방어
- **LLM은 '유도', 규칙은 '안전망'**

안전 불변식: 레벨은 '탐지 적극성'만 조절한다. NFR-01(비파괴)는 레벨과 무관하게 항상 고정.

① 실증 — 레벨이 미탐을 구제

실제 Gemini 판단에 레벨 적용 (VulnBoard) — Gemini가 skip한 4개 파라미터 처리

4
Gemini가 skip (sort·password·attach)

1
Lv.2에서 구제 (위험: sort)

4
Lv.3에서 전부 구제

파라미터	Lv1 (정밀)	Lv2 (균형)	Lv3 (보수적)
sort (위험)	skip	둘 다 ✓	둘 다 ✓
password	skip	skip	둘 다 ✓
content (비위험)	LLM 그대로	LLM 그대로	둘 다 ✓

→ 레벨을 올릴수록 LLM이 놓친 것을 구조적으로 다시 잡는다 (미탐↓, 스캔량↑).

① 실증 — 레벨이 미탐을 구제

실제 Gemini 판단에 레벨 적용 (VulnBoard) — Gemini가 skip한 4개 파라미터 처리

FR-02 · 도구 선택 (레벨 1)

ID	선택된 스캐너	우선순위	판단 근거
ip-001	sqlmap,dalfox	sqlmap → dalfox	대상 파라미터 'q'는 검색어 용도로 DB 조회 및 응답 반사 가능성이 모두 높아 SQL Injection과 Reflected XSS 스캐너를 순서대로 적용합니다.
ip-002	sqlmap	sqlmap	category 파라미터는 DB 조회에 사용될 가능성이 높아 sqlmap을 선정하였으며, XSS의 명확한 반사 증적이 없어 dalfox는 생략합니다.
ip-003	**skip**		대상 파라미터 'sort'는 쇼핑몰의 정렬 보여 SQL Injection이나 XSS 취약점 발
ip-004	sqlmap	sqlmap	대상 파라미터가 product_id로 DB 조회 해 sqlmap을 선정하였습니다. XSS의 경 성이 낮아 제외했습니다.

FR-04 · 판단 (레벨 1)

- 확정(confirmed) **4건** / 미확인 0건

ID	유형	상태	신뢰도	증거
ip-001	sqli	✅ confirmed	0.95	db_error, boolean_diff, union_data
ip-001	xss	✅ confirmed	0.95	reflection, executable_context
ip-002	sqli	✅ confirmed	0.85	db_error, boolean_diff
ip-004	sqli	✅ confirmed	0.9	other

② FR-01 ~ FR-04 통합 실행

- **크롤링 → 도구 선택 → 스캔 → 판단을 1회로 완주 (레벨을 맨 처음 선택)**
- 각 단계 출력은 JSON으로 다음 단계에 전달 (01_crawl → 02_select → 03_scan → 04_judge)

FR-02 · 도구 선택 (레벨 1)

ID	선택된 스캐너	우선순위	판단 근거
ip-001	sqlmap,dalfox	sqlmap → dalfox	대상 파라미터 'q'는 검색어 용도로 DB 조회 및 응답 반사 가능성이 모두 높아 SQL Injection과 Reflected XSS 스캐너를 순서대로 적용합니다.
ip-002	sqlmap	sqlmap	category 파라미터는 DB 조회에 사용될 가능성이 높아 sqlmap을 선정하였으며, XSS의 확한 반사 증적이 없어 dalfox는 생략합니다.
ip-003	**skip**		대상 파라미터 'sort'는 쇼핑몰의 정렬 방식을 지정하는 순수 표시용 및 정적 설정 값으로 보여 SQL Injection이나 XSS 취약점 발생 가능성이 낮아 스캐너를 생략합니다.
ip-004	sqlmap	sqlmap	대상 파라미터가 product_id로 DB 조회에 사용될 가능성이 높아 SQL Injection 탐지를 해 sqlmap을 선정하였습니다. XSS의 경우 body 내의 hidden 파라미터로 값의 반사 가성이 낮아 제외했습니다.

FR-04 · 판단 (레벨 1)

- 확정(confirmed) **4건** / 미확인 0건

ID	유형	상태	신뢰도	증거
ip-001	sqli	✅ confirmed	0.95	db_error, boolean_diff, union_data
ip-001	xss	✅ confirmed	0.95	reflection, executable_context
ip-002	sqli	✅ confirmed	0.85	db_error, boolean_diff
ip-004	sqli	✅ confirmed	0.9	other

FR-04 출력 = 구조화 JSON (status · 신뢰도 · 증거 · 판단근거) → 리포트·추적에 그대로 사용. 오탐·미탐 없이 확정 3건.

② 문제 — 실행 시간이 오래 걸림

- **스캔(FR-03)이 병목 — 전체 시간의 대부분을 차지**
- 원인: 스캐너가 요청을 수백 번 전송 (dalfox 파라미터당 ~467요청)
- 안전을 위해 단일 스레드·요청 간 200ms 지연(NFR-02)으로 일부러 천천히

~98초	~2.5분	수시간
dalfox · 파라미터 1개	입력지점 1개 (둘 다)	전체 사이트 (제한 없이)

추가 관찰: 동적 스캔은 같은 입력이라도 실행시간이 변동한다 (요청별 서버 응답시간 편차, sqlmap의 비취약 판정 탐색). **단, 발견 결과는 일관됨 — '결과는 재현되지만 시간은 변동'.**

② 해결 방향 — 시간 단축 전략

중복 제거

구조 동일 엔드포인트는 1번만 스캔

스마트 라우팅

LLM이 관련 스캐너만 선택 (FR-02)

탐지 레벨

위험 기반으로 깊이 조절

병렬화

안전 범위 내 동시 스캔

차주 계획

- FR-02 중복 제거 내장 — LLM 호출 수 절감
- FR-05 리포트까지 연결 — 완전한 1회 진단 완주
- 모델별 비교 실험

요약: 미탐/오탐 → '위험 기반 탐지 레벨'로 해결 · FR-01~04 통합 실행 완료 · 속도 병목 식별 후 중복제거로 50% 단축